Qbatch Interview Preparation

Associate Software Engineer | Onsite | Faisalabad | 15 June 2026 | 11:30 AM

DSA • OOP • SQL • Arrays • LinkedList • Stack • BST • Django • FYP • Coding Problems

WHAT TO EXPECT — Based on 3 Colleagues + WAMO

Topic	What They Asked	Difficulty
Arrays	Multi-D vs 1D array, reverse array	Easy
SQL	SELECT FROM WHERE ORDER BY, JOIN queries, subqueries, schema design	Medium
DB Schema	ER diagram, relations, attributes, foreign/primary keys	Medium
DSA	Stack basics, BST traversals, complexity	Medium
Django	Basic questions about models, views, ORM	Easy-Medium
FYP	Your contribution, architecture, what you built	Easy
OOP	Encapsulation, inheritance, polymorphism	Easy
Problem Solving	Digital root, sort stack using stack, encode/decode string	Medium
LinkedList	Fast-slow pointer, sum of two numbers as linked list	Medium

SECTION 1 — ARRAYS & MULTI-DIMENSIONAL

Q1. What is the difference between 1D and multi-dimensional arrays?

A: 1D array: single row of elements accessed by one index. arr[i]. 2D array: matrix of rows and columns accessed by two indices. arr[i][j]. Multi-D arrays are stored in row-major order in memory — all elements of row 0, then row 1, etc.

■ **Access: O(1) for both. Space: O(n) for 1D, O(n*m) for 2D.**

```
// 1D Array
int arr[5] = {1,2,3,4,5};
cout << arr[2]; // 3

// 2D Array
int matrix[3][3] = {{1,2,3},{4,5,6},{7,8,9}};
cout << matrix[1][2]; // 6

// Memory layout (row-major):
// matrix[0][0], matrix[0][1], matrix[0][2],
// matrix[1][0], matrix[1][1], matrix[1][2]...
```

Q2. Reverse a 1D array in-place.

A: Use two pointers — left starting at 0, right at n-1. Swap and move both inward until they meet.

■ **Time: O(n), Space: O(1)**

```
void reverseArray(int arr[], int n) {
  int left = 0, right = n - 1;
  while (left < right) {
    int temp = arr[left];
    arr[left] = arr[right];
    arr[right] = temp;
    left++;
    right--;
  }
}
// Input:  [1, 2, 3, 4, 5]
// Output: [5, 4, 3, 2, 1]
```

Q3. Reverse a 2D array (rotate/transpose).

A: To reverse each row: apply 1D reverse to each row. To transpose: swap arr[i][j] with arr[j][i].

■ **Time: O(n*m), Space: O(1)**

```
// Reverse each row of 2D array
void reverse2D(int arr[][3], int rows, int cols) {
  for (int i = 0; i < rows; i++) {
    int left = 0, right = cols - 1;
    while (left < right) {
      swap(arr[i][left], arr[i][right]);
      left++; right--;
    }
  }
}
```

Q4. Find maximum element in array.

A: Initialize max as first element. Iterate through rest, update max if current > max.

■ **Time: O(n), Space: O(1)**

```
int findMax(int arr[], int n) {
  int maxVal = arr[0];
  for (int i = 1; i < n; i++) {
    if (arr[i] > maxVal) maxVal = arr[i];
  }
  return maxVal;
}
```

Q5. Two Sum — find two numbers that add to target.

A: Use HashMap. For each element, check if (target - element) exists in map. If yes, found the pair.

■ **Time: O(n), Space: O(n) — beats brute force O(n²)**

```
// Using unordered_map (C++)
pair twoSum(int arr[], int n, int target) {
  unordered_map seen;
  for (int i = 0; i < n; i++) {
    int complement = target - arr[i];
    if (seen.count(complement))
      return {seen[complement], i};
    seen[arr[i]] = i;
  }
  return {-1,-1};
}
// Input: [2,7,11,15], target=9
// Output: {0,1} (indices of 2 and 7)
```

SECTION 2 — LINKED LIST PROBLEMS

Q6. Fast-Slow pointer — detect cycle in linked list.

A: Use two pointers. Slow moves 1 step, fast moves 2 steps. If they meet, cycle exists. Floyd's algorithm.

■ **Time: O(n), Space: O(1)**

```
bool hasCycle(Node* head) {
  Node* slow = head;
  Node* fast = head;
  while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) return true;
  }
  return false;
}
```

Q7. Add two numbers represented as linked lists (1->2->3->4 + 5->6->7->8).

A: Traverse both lists simultaneously. Add digits + carry. Store result digit in new node. Handle remaining carry at end.

■ **Time: O(max(m,n)), Space: O(max(m,n))**

```
Node* addTwoNumbers(Node* l1, Node* l2) {
  Node dummy(0);
  Node* curr = &dummy;
  int carry = 0;
  while (l1 || l2 || carry) {
    int sum = carry;
    if (l1) { sum += l1->data; l1 = l1->next; }
    if (l2) { sum += l2->data; l2 = l2->next; }
    carry = sum / 10;
    curr->next = new Node(sum % 10);
    curr = curr->next;
  }
  return dummy.next;
}
// 1->2->3->4  +  5->6->7->8
// = 6->9->1->3  (reversed digit order)
```

Q8. Reverse a linked list.

A: Use three pointers: prev=NULL, curr=head, next=NULL. In each step: save next, reverse curr->next to prev, advance both.

■ **Time: O(n), Space: O(1)**

```
Node* reverse(Node* head) {
  Node* prev = NULL;
  Node* curr = head;
  while (curr != NULL) {
    Node* next = curr->next; // save next
    curr->next = prev;       // reverse link
    prev = curr;             // advance prev
    curr = next;             // advance curr
  }
  return prev; // new head
}
```

Q9. Find middle of linked list.

A: Fast-slow pointer. When fast reaches end, slow is at middle.

■ **Time: O(n), Space: O(1)**

```
Node* findMiddle(Node* head) {
  Node* slow = head;
  Node* fast = head;
  while (fast != NULL && fast->next != NULL) {
    slow = slow->next;
    fast = fast->next->next;
  }
  return slow; // middle node
}
// 1->2->3->4->5 returns node 3
```

SECTION 3 — STACK PROBLEMS

Q10. Sort a stack using another stack.

A: Pop from stack1. While stack2 top > popped element, move stack2 top back to stack1. Push popped to stack2. Repeat.

■ **Time: O(n²), Space: O(n)**

```
void sortStack(stack& s) {
  stack temp;
  while (!s.empty()) {
    int curr = s.top(); s.pop();
    while (!temp.empty() && temp.top() > curr) {
      s.push(temp.top());
      temp.pop();
    }
    temp.push(curr);
  }
  // copy back
  while (!temp.empty()) {
    s.push(temp.top());
    temp.pop();
  }
}
// Input:  [3,1,4,2,5]
// Output: [1,2,3,4,5] (sorted)
```

Q11. Balanced parentheses using stack.

A: Push opening brackets. On closing bracket, check if stack top matches. If mismatch or empty, return false. Valid if stack empty at end.

■ **Time: O(n), Space: O(n)**

```
bool isBalanced(string s) {
  stack st;
  for (char c : s) {
    if (c=='(' || c=='[' || c=='{') st.push(c);
    else {
      if (st.empty()) return false;
      char top = st.top(); st.pop();
      if (c==')' && top!='(') return false;
      if (c==']' && top!='[') return false;
      if (c=='}' && top!='{') return false;
    }
  }
  return st.empty();
}
// "({[]})" -> true
// "([)]"   -> false
```

SECTION 4 — BINARY SEARCH TREE & TRAVERSALS

Q12. BST Traversals — InOrder, PreOrder, PostOrder.

A: InOrder (Left-Root-Right): gives sorted output. PreOrder (Root-Left-Right): copy tree. PostOrder (Left-Right-Root): delete tree.

■ **Time: O(n) all traversals, Space: O(h) where h=height**

```
void inOrder(Node* root) {   // Left Root Right
  if (!root) return;
  inOrder(root->left);
  cout << root->data << " ";
  inOrder(root->right);
}
void preOrder(Node* root) {  // Root Left Right
  if (!root) return;
  cout << root->data << " ";
  preOrder(root->left);
  preOrder(root->right);
}
void postOrder(Node* root) { // Left Right Root
  if (!root) return;
  postOrder(root->left);
  postOrder(root->right);
  cout << root->data << " ";
}
// Tree:    5
//         / //        2   7
//       / //      1   3
// InOrder:   1 2 3 5 7 (SORTED)
// PreOrder:  5 2 1 3 7
// PostOrder: 1 3 2 7 5
```

Q13. BST Complexity Summary.

A: Average case O(log n) for balanced tree. Worst case O(n) for skewed tree (like sorted input).

```
// Average (balanced):  Worst (skewed):
// Insert:  O(log n)        O(n)
// Search:  O(log n)        O(n)
// Delete:  O(log n)        O(n)
// Space:   O(n)            O(n)

// AVL tree guarantees O(log n) always
// by auto-balancing after each insert/delete
```

Q14. Height of binary tree.

A: Recursively find height of left and right subtrees. Return 1 + max of both. Base case: null = 0.

■ **Time: O(n), Space: O(h)**

```
int height(Node* root) {
  if (root == NULL) return 0;
  int left  = height(root->left);
  int right = height(root->right);
  return 1 + max(left, right);
}
```

SECTION 5 — SQL QUERIES

Q15. Basic SELECT with WHERE and ORDER BY.

A: SELECT columns FROM table WHERE condition ORDER BY column ASC/DESC LIMIT n.

```
-- Find all employees with salary > 50000
SELECT name, salary
FROM employees
WHERE salary > 50000
ORDER BY salary DESC;

-- Top 3 highest paid
SELECT name, salary
FROM employees
ORDER BY salary DESC
LIMIT 3;

-- Filter by date (last 1 month)
SELECT * FROM payments
WHERE created_at >= DATE_SUB(NOW(), INTERVAL 1 MONTH);
```

Q16. JOIN queries — table tournament scores.

A: INNER JOIN returns matching rows from both tables. Use ON to specify join condition.

```
-- Schema: players(id, name), scores(id, player_id, score, match_date)

-- Max score per player
SELECT p.name, MAX(s.score) as max_score
FROM players p
JOIN scores s ON p.id = s.player_id
GROUP BY p.id, p.name
ORDER BY max_score DESC;

-- Scores from last 1 month with player name
SELECT p.name, s.score, s.match_date
FROM players p
JOIN scores s ON p.id = s.player_id
WHERE s.match_date >= DATE_SUB(NOW(), INTERVAL 1 MONTH)
ORDER BY s.score DESC;

-- Player with highest total score
SELECT p.name, SUM(s.score) as total
FROM players p
JOIN scores s ON p.id = s.player_id
GROUP BY p.id, p.name
ORDER BY total DESC
LIMIT 1;
```

Q17. Subqueries — second highest score.

A: Find max excluding the overall max using nested SELECT.

```
-- Second highest score
SELECT MAX(score) as second_highest
FROM scores
WHERE score < (SELECT MAX(score) FROM scores);

-- Students scoring above average
SELECT name, score
FROM students
WHERE score > (SELECT AVG(score) FROM students);

-- Find duplicate records
SELECT player_id, COUNT(*) as count
FROM scores
GROUP BY player_id
HAVING COUNT(*) > 1;
```

Q18. Design schema for tournament. Write ER and queries.

A: Entities: Player, Match, Score. Player has id, name. Match has id, date, venue. Score has id, player_id, match_id, points.

```
-- Schema Design
CREATE TABLE players (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  email VARCHAR(100) UNIQUE
);

CREATE TABLE matches (
  id INT PRIMARY KEY AUTO_INCREMENT,
  match_date DATE,
  venue VARCHAR(100)
);

CREATE TABLE scores (
  id INT PRIMARY KEY AUTO_INCREMENT,
  player_id INT REFERENCES players(id),
  match_id  INT REFERENCES matches(id),
  points    INT,
  created_at DATETIME DEFAULT NOW()
);

-- All matches with player scores
SELECT p.name, m.match_date, s.points
FROM scores s
JOIN players p ON s.player_id = p.id
JOIN matches m ON s.match_id  = m.id
ORDER BY m.match_date DESC;
```

Q19. WHERE vs HAVING difference.

A: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER GROUP BY. WHERE cannot use aggregate functions.

```
-- WHERE: filter before group
SELECT dept, COUNT(*)
FROM employees
WHERE salary > 30000       -- filters rows first
GROUP BY dept;

-- HAVING: filter after group
SELECT dept, AVG(salary) as avg_sal
FROM employees
GROUP BY dept
HAVING AVG(salary) > 50000; -- filters groups
```

SECTION 6 — OOP CONCEPTS

Q20. 4 Pillars of OOP with real examples.

A: Encapsulation=data hiding (private fields). Inheritance=child extends parent. Polymorphism=same method different behavior. Abstraction=hide complexity.

```
// Encapsulation
class BankAccount {
  private: double balance; // hidden
  public:
    void deposit(double amt) { balance += amt; }
    double getBalance() { return balance; }
};

// Inheritance
class Animal { public: virtual void speak() {} };
class Dog : public Animal {
  public: void speak() { cout << "Woof"; }
};

// Polymorphism (runtime)
Animal* a = new Dog();
a->speak(); // prints "Woof" not Animal

// Abstraction
class Shape {
  public: virtual double area() = 0; // pure virtual
};
class Circle : public Shape {
  double r;
  public: double area() { return 3.14*r*r; }
};
```

Q21. What is the difference between overloading and overriding?

A: Overloading: same name, different parameters, same class, compile-time. Overriding: child redefines parent method with same signature, runtime.

```
// Overloading (compile-time)
class Math {
  int add(int a, int b) { return a+b; }
  double add(double a, double b) { return a+b; } // same name diff params
};

// Overriding (runtime)
class Animal { virtual void speak() { cout << "..."; } };
class Cat : public Animal {
  void speak() override { cout << "Meow"; } // redefines parent
};
```

SECTION 7 — CODING PROBLEMS (WAMO + Expected)

Q22. Digital Root of a number.

A: Repeatedly sum digits until single digit. Formula: if n==0 return 0, else return 1 + (n-1)%9.

■ **Time: O(log n), Space: O(1)**

```
// Iterative approach
int digitalRoot(int n) {
  while (n >= 10) {
    int sum = 0;
    while (n > 0) {
      sum += n % 10;
      n /= 10;
    }
    n = sum;
  }
  return n;
}

// O(1) Math formula
int digitalRootFast(int n) {
  if (n == 0) return 0;
  return 1 + (n - 1) % 9;
}
// Input: 9875 -> 9+8+7+5=29 -> 2+9=11 -> 1+1=2
// Output: 2
```

Q23. Encode string — vowels to next vowel (a->e, e->i, i->o, o->u, u->a), toggle case.

A: Map each vowel to next vowel. Toggle case using XOR with 32 (converts between upper and lower).

■ **Time: O(n), Space: O(n)**

```
string encode(string s) {
  string vowels = "aeiouAEIOU";
  string nextV  = "eiouaEIOUA";
  string result = "";
  for (char c : s) {
    int pos = vowels.find(c);
    if (pos != string::npos) {
      result += nextV[pos]; // next vowel
    } else if (isalpha(c)) {
      result += (char)(c ^ 32); // toggle case
    } else {
      result += c;
    }
  }
  return result;
}
// "Hello World" -> "Hille Wirld" (approx)
```

Q24. Count and say '0' in string — take 0s, describe count.

A: Run-length encoding: count consecutive same characters and output count+character.

■ **Time: O(n), Space: O(n)**

```
string countAndSay(string s) {
  string result = "";
  int i = 0;
  while (i < s.length()) {
    char curr = s[i];
    int count = 0;
    while (i < s.length() && s[i] == curr) {
      count++;
      i++;
    }
    result += to_string(count) + curr;
  }
  return result;
}
// "000111" -> "3031" (three 0s, three 1s)
// "0011100" -> "2031020"
```

Q25. Fibonacci — iterative (not recursive).

A: Use two variables a=0 and b=1. Each step: temp=a+b, a=b, b=temp. Iterative avoids O(2^n) stack overhead.

■ **Time: O(n), Space: O(1)**

```
int fibonacci(int n) {
  if (n <= 1) return n;
  int a = 0, b = 1;
  for (int i = 2; i <= n; i++) {
    int temp = a + b;
    a = b;
    b = temp;
  }
  return b;
}
// fib(0)=0, fib(1)=1, fib(2)=1,
// fib(3)=2, fib(4)=3, fib(5)=5, fib(10)=55
```

Q26. Check if string is palindrome.

A: Two pointers from both ends. Compare characters. Skip non-alphanumeric if needed.

■ **Time: O(n), Space: O(1)**

```
bool isPalindrome(string s) {
  int left = 0, right = s.length() - 1;
  while (left < right) {
    if (s[left] != s[right]) return false;
    left++;
    right--;
  }
  return true;
}
// "racecar" -> true
// "hello"   -> false
```

Q27. Find duplicates in array.

A: Use unordered_set. Add each element, if already exists it's a duplicate.

■ **Time: O(n), Space: O(n)**

```
vector findDuplicates(int arr[], int n) {
  unordered_set seen;
  vector duplicates;
  for (int i = 0; i < n; i++) {
    if (seen.count(arr[i]))
      duplicates.push_back(arr[i]);
    else
      seen.insert(arr[i]);
  }
  return duplicates;
}
// Input:  [1,2,3,2,4,3,5]
// Output: [2, 3]
```

Q28. Binary Search.

A: Divide search space in half each step. Only works on SORTED array.

■ **Time: O(log n), Space: O(1)**

```
int binarySearch(int arr[], int n, int target) {
  int left = 0, right = n - 1;
  while (left <= right) {
    int mid = left + (right - left) / 2;
    if (arr[mid] == target) return mid;
    else if (arr[mid] < target) left = mid + 1;
    else right = mid - 1;
  }
  return -1; // not found
}
// Input: [1,3,5,7,9,11], target=7
// Returns index 3
```

SECTION 8 — DJANGO QUESTIONS

Q29. What is Django ORM? Basic queries.

A: ORM = Object Relational Mapper. Write Python instead of SQL. Django translates to SQL automatically.

```
# Basic ORM queries
Property.objects.all()              # SELECT *
Property.objects.filter(city='LHR') # WHERE city='LHR'
Property.objects.get(id=1)          # WHERE id=1 (one result)
Property.objects.create(title='Flat') # INSERT
Property.objects.filter(id=1).update(title='New') # UPDATE
Property.objects.filter(id=1).delete() # DELETE

# Order and limit
Property.objects.order_by('-created_at')[:10]

# Q objects (OR query)
from django.db.models import Q
Property.objects.filter(Q(owner=user) | Q(tenant=user))

# select_related (prevents N+1)
Agreement.objects.select_related('property', 'tenant').all()
```

Q30. What is JWT and how does it work in Django?

A: JWT = JSON Web Token. User logs in, server returns access+refresh token. Client sends access token in Authorization header. Server validates without DB lookup.

```
# Login -> returns tokens
POST /auth/login/
{ "username": "faizan", "password": "pass" }
-> { "access": "eyJ...", "refresh": "eyJ..." }

# Use token in requests
Authorization: Bearer eyJ...

# Django validates via JWTAuthentication middleware
# No DB lookup needed - signature verified from SECRET_KEY

# settings.py
REST_FRAMEWORK = {
  'DEFAULT_AUTHENTICATION_CLASSES': (
    'rest_framework_simplejwt.authentication.JWTAuthentication',
  )
}
```

SECTION 9 — FYP ANSWERS (HealthMate)

Q31. Tell me about your FYP.

A: HealthMate is a mobile health monitoring application. Users track lifestyle, genetic, and wearable data. Provides real-time health risk scores using XGBoost model.

```
Tech Stack: React Native + Expo (frontend)
             Flask (backend API)
             MySQL (database)
             XGBoost (ML model)
             Docker (containerization)

Architecture:
- Mobile app collects health data
- Sends to Flask REST API
- API runs XGBoost prediction
- Returns risk score in < 5 seconds
- MySQL stores 10+ health metrics per session

My contribution:
- Flask backend with JWT authentication
- XGBoost model integration
- MySQL schema design
- Wearable device data integration
- Docker containerization
```

Q32. What challenges did you face in FYP?

A: Three main challenges: integrating wearable device data formats, optimizing XGBoost predictions to respond under 5 seconds, and handling real-time data synchronization between mobile and backend.

```
Challenge 1: Wearable data inconsistency
  - Different devices output different formats
  - Solution: Normalization layer in Flask before model input

Challenge 2: Response time > 5 seconds
  - XGBoost model loading on every request was slow
  - Solution: Load model once on server startup, reuse

Challenge 3: Real-time sync
  - MySQL writes conflicting with reads
  - Solution: Proper transaction management
```

SECTION 10 — COMPLEXITY CHEAT SHEET

Operation	Data Structure	Time Complexity	Space
Access by index	Array	O(1)	O(n)
Search	Array (unsorted)	O(n)	O(1)
Search	Array (sorted/binary)	O(log n)	O(1)
Insert/Delete at head	LinkedList	O(1)	O(1)
Insert/Delete at tail	LinkedList	O(n) or O(1) with tail ptr	O(1)
Push/Pop	Stack	O(1)	O(n)
Enqueue/Dequeue	Queue	O(1)	O(n)
Get/Put	HashMap	O(1) avg	O(n)
Insert/Search/Delete	BST (balanced)	O(log n)	O(n)
Insert/Search/Delete	BST (worst/skewed)	O(n)	O(n)
Insert (heapify)	Heap	O(log n)	O(n)
Get min/max	Heap	O(1)	O(n)

Bubble/Selection/Insertion Sort	Array	$O(n^2)$	O(1)
Merge Sort	Array	O(n log n)	O(n)
Quick Sort	Array	O(n log n) avg	O(log n)
BFS/DFS	Graph	O(V+E)	O(V)

LAST 30 MINS — READ THIS BEFORE INTERVIEW

✓ InOrder BST = sorted output (Left → Root → Right)

✓ Reverse array = two pointers left/right, swap, move inward — O(n) time O(1) space

✓ Sort stack = use second stack, insert in sorted order — $O(n^2)$ time

✓ Digital root = 1 + (n-1)%9 for n>0 — O(1) formula

✓ Fast-slow pointer = cycle detection, finding middle — O(n) time O(1) space

✓ Two sum = HashMap for O(n) instead of brute force $O(n^2)$

✓ WHERE = filter before GROUP BY. HAVING = filter after GROUP BY

✓ JOIN = INNER (matching only), LEFT (all left + matching right)

✓ Primary key = unique + not null. Foreign key = references another table's PK

✓ Encapsulation = private + getters. Inheritance = extends. Polymorphism = override

✓ JWT = stateless auth. Token in Authorization: Bearer header. No DB lookup

✓ FYP: React Native + Flask + MySQL + XGBoost + Docker. Your contribution = backend + ML

✓ Be confident. If you don't know, say 'I'd approach it this way...' and show thinking

✓ Arrive 10 mins early. Sitara Techno Park, Main E-Canal Expy Block-B, Peoples Colony #1